

Technical Specification: DGOJ Synchronous Response Validator

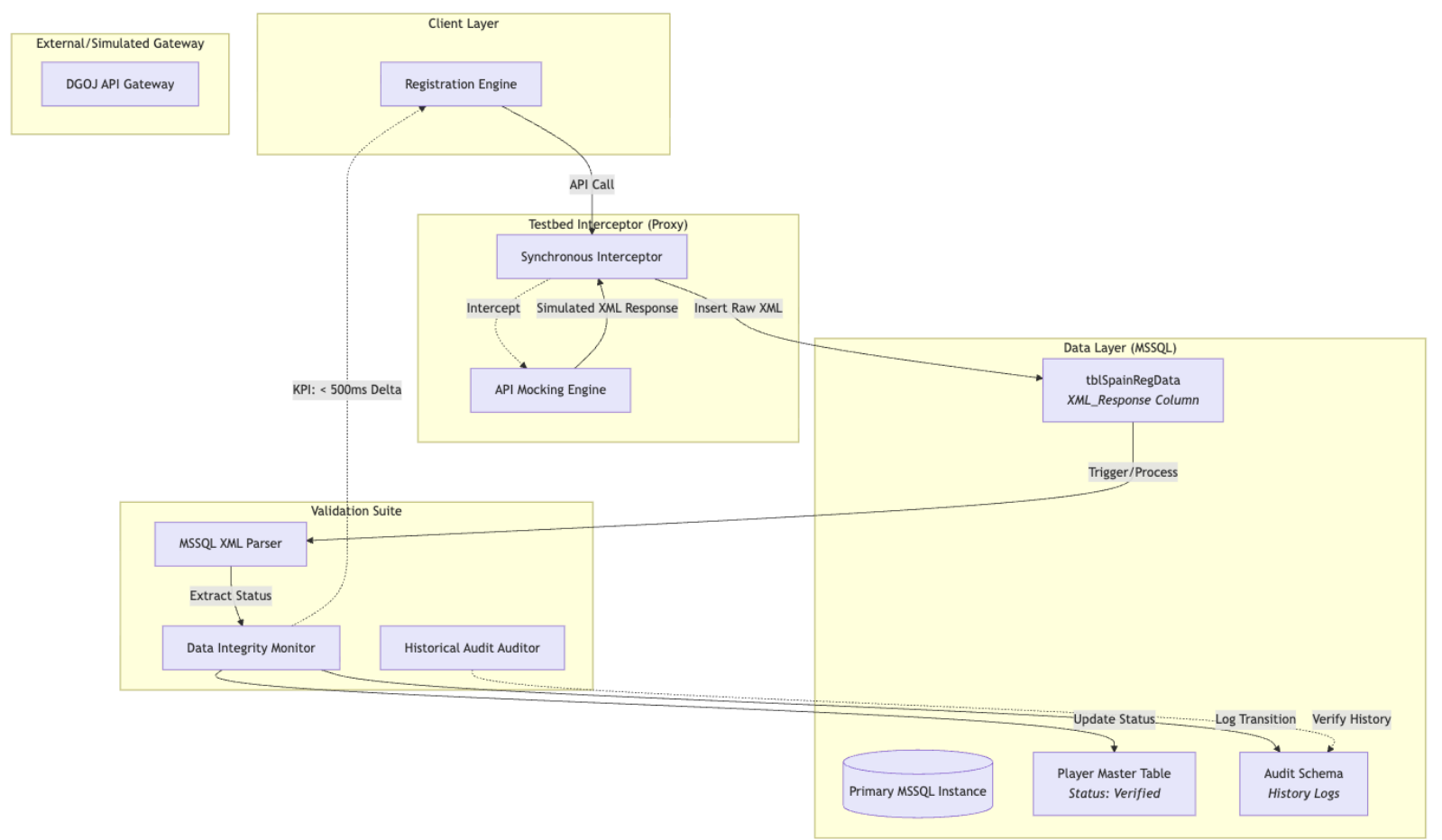
1. Overview

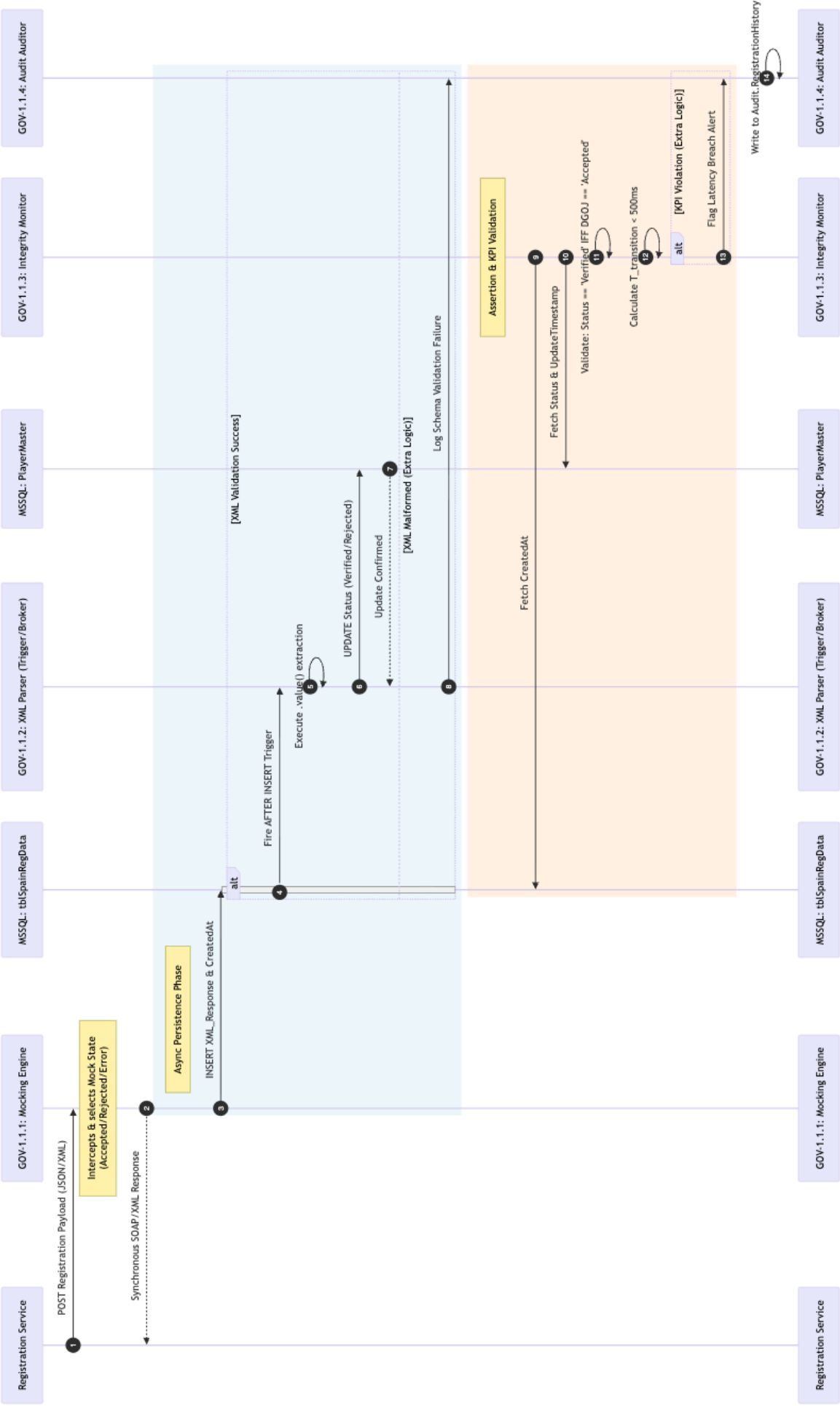
This document outlines the architecture and technical requirements for the **Automated Spanish DGOJ Synchronous Response Validator**. This system serves as a regulatory "testbed" to ensure that the integration between internal registration services and the Spanish Directorate General for the Regulation of Gambling (DGOJ) maintains 100% data integrity and meets strict latency requirements.

1.1 Objective

To intercept registration API calls, simulate regulatory responses, and validate that the internal MSSQL state transitions (from raw XML to 'Verified' status) occur accurately and within a $\Delta T_{\text{Delta}} < 500\text{ms}$ threshold.

2. System Architecture





3. Component Specifications

3.1 Synchronous Interceptor & Mocking Engine (GOV-1.1.1)

The Interceptor acts as a transparent proxy. In the testbed environment, it reroutes outbound DGOJ requests to the internal **Mocking Engine**.

- **Input:** Registration Payload (JSON/XML).
- **Output:** Simulated SOAP/XML response.
- **Mock States:** Must support `Accepted`, `Rejected`, and `Error`.

3.2 MSSQL XML Parser (GOV-1.1.2)

This component monitors the `XML_Response` column in `tblSpainRegData`. It is responsible for deserializing the DGOJ response and extracting the status code.

- **Technology:** T-SQL XML Data Type methods (`.value()`, `.nodes()`).
- **Logic:**
- SQL

None

```
-- Example extraction logic
SELECT
    XML_Response.value('/DGOJResponse/Status)[1]', 'VARCHAR(20)') AS DGOJStatus
FROM tblSpainRegData
WHERE RequestID = @RequestID;
```

3.3 Data Integrity Monitor (GOV-1.1.3)

An assertion engine that compares the expected mock state against the actual resulting state in the **Player Master Table**.

- **Verification:** Ensure `Status = 'Verified'` if and only if `DGOJStatus = 'Accepted'`.
- **Latency Check:** Calculates the delta between `RequestTimestamp` and `UpdateTimestamp`.

3.4 Historical Audit Auditor (GOV-1.1.4)

A background process that ensures every interaction is logged in the `Audit.RegistrationHistory` schema for regulatory reporting.

4. Functional Requirements Mapping

ID	Component	Requirement

GOV-1.1.1	API Mocking Engine	Return simulated <code><Status>Accepted</Status></code> response upon interception.
GOV-1.1.2	MSSQL XML Parser	Parse the <code>XML_Response</code> column in <code>tblSpainRegData</code> in real-time.
GOV-1.1.3	SQL Monitor	Assert player status transition to <code>Verified</code> .
GOV-1.1.4	Audit Auditor	Verify a complete history of attempts exists in the audit schema.

5. Success Metrics (KPIs)

- **Mapping Accuracy:** 100% . No mismatch between DGOJ XML status and internal database flags is permitted.
- **State Transition Latency:**
 $T_{\text{transition}} = T_{\text{DB_Update}} - T_{\text{API_Receipt}} < 500\text{ms}$
- **Parsing Success:** Zero failures in XML schema validation within `tblSpainRegData`.

6. Implementation Notes

[!IMPORTANT]

Performance Optimization: To meet the 500ms KPI, avoid heavy cursor-based logic. It is recommended to use an **AFTER INSERT Trigger** on `tblSpainRegData` or an optimized Service Broker queue to handle the downstream updates to the Player Master Table.

6.1 Database Schema (Snippet)

- **Table:** `tblSpainRegData`
 - `RegID` (PK, int)
 - `PlayerID` (FK, int)
 - `XML_Response` (XML)
 - `CreatedAt` (datetime2)
- **Table:** `PlayerMaster`
 - `PlayerID` (PK, int)
 - `Status` (nvarchar) — Expected: 'Verified', 'Pending', 'Rejected'.